

gPCD Geometry Dynamics

May 18, 2026

1 A first collision function

We need to have a relationship between velocity in (v_{in}) (just prior to the collision), the velocity at any degree of penetration (v_p), the total circle area (A_T) and penetration area (A_P). In an *instantaneous, purely geometric* system is that the ratio of incoming velocity (the velocity just before the collision occurs) to penetration velocity is proportional to the ratio of total area to penetration area.

$$\frac{v_{in}}{v_p} \propto \frac{A_T}{A_P} \quad (1)$$

The proportionality factor is the force with which the particle resists collision at depth of penetration (α) or repulsion force ($F_R(\alpha)$) being an increasing parabolic function. So,

$$\frac{v_{in}}{v_p} = F_R(\alpha) \frac{A_T}{A_P} \quad (2)$$

and,

$$v_{in} \frac{1}{F_R(\alpha)} \frac{A_P}{A_T} = v_p \quad (3)$$

2 A second collision function

A second collision function is that the change in velocity is proportional to the penetration area,

$$(v_{in} - v_p) = P A_P \quad (4)$$

So that

$$v_p = v_{in} - P A_P \quad (5)$$

At the turnaround point, the penetration velocity is zero:

$$0 = v_{in} - P A_0 \quad (6)$$

so the zero-velocity penetration area can be predicted from incoming velocity:

$$A_0 = \frac{v_{in}}{P} \quad (7)$$

3 Equivalence of the two functions

This section derives the algebraic force factor needed to make the first collision function and second collision function describe the same velocity curve.

The first function gives:

$$v_p = v_{in} \frac{1}{F_R(\alpha)} \frac{A_P}{A_T} \quad (8)$$

The second function gives:

$$v_p = v_{in} - P A_P \quad (9)$$

For the two functions to be equivalent, the two expressions for v_p must be equal:

$$v_{in} \frac{1}{F_R(\alpha)} \frac{A_P}{A_T} = v_{in} - P A_P \quad (10)$$

Solving for the resistance function gives:

$$F_R(\alpha) = \frac{v_{in} A_P}{A_T (v_{in} - P A_P)} \quad (11)$$

Using the turnaround relation $A_0 = v_{in}/P$, this can also be written as:

$$F_R(\alpha) = \frac{A_P A_0}{A_T (A_0 - A_P)} \quad (12)$$

Thus the first function and second function can describe the same collision curve if $F_R(\alpha)$ is chosen consistently. As A_P approaches A_0 , the denominator approaches zero, so $F_R(\alpha)$ grows without bound. This corresponds to the zero-velocity turnaround point.

4 Force factor

This section gives a physical force law that can be used to compute the turnaround depth by energy balance. This is different from the algebraic equivalence factor in Section 3. The Section 3 factor is the exact function needed to make two velocity equations equivalent. The force law here describes how collision resistance grows with penetration depth.

Assume the repulsive force is an increasing parabolic function of penetration depth:

$$F(\alpha) = Q \alpha^2 \quad (13)$$

where Q is the stiffness factor.

At the zero-velocity turnaround point, incoming normal kinetic energy has been stored by the repulsive force:

$$\frac{1}{2} \mu v_{in}^2 = \int_0^{\alpha_0} Q \alpha^2 d\alpha \quad (14)$$

Evaluating the integral:

$$\frac{1}{2} \mu v_{in}^2 = \frac{Q \alpha_0^3}{3} \quad (15)$$

Solving for turnaround penetration depth:

$$\alpha_0 = \left(\frac{3\mu v_{in}^2}{2Q} \right)^{1/3} \quad (16)$$

The zero-velocity penetration area is then:

$$A_0 = A(\alpha_0) \quad (17)$$

Thus Section 3 gives the algebraic force factor for equivalence, while this section gives a physical force law for predicting the turnaround depth.

5 Impact on starting in a collision

When particles start in a collision, the configured velocity is interpreted as the incoming velocity before turnaround. The current overlap is assumed to be on the rebound side of the collision history.

With the physical force law, the zero-velocity penetration depth is no longer only a configured fraction. It can be predicted from the incoming velocity and the stiffness factor Q :

$$\alpha_0 = \left(\frac{3\mu v_{in}^2}{2Q} \right)^{1/3} \quad (18)$$

and:

$$A_0 = A(\alpha_0) \quad (19)$$

For a starting overlap with current penetration area A_P :

- If $A_P < A_0$, the starting state is physically valid and is on the rebound path.
- If $A_P = A_0$, the starting state is exactly at the zero-velocity turnaround point.
- If $A_P > A_0$, the starting overlap is deeper than the incoming velocity and stiffness factor could physically produce.

The last case requires an explicit policy. The simulation can:

- promote A_0 to the current penetration area, as the current Python model does,
- reject the configuration as physically impossible,
- or infer the larger incoming velocity required to reach the current penetration area.

Thus, for starting collisions, the force-law model replaces an arbitrary `zero_velocity_overlap_fraction` with a predicted turnaround point based on incoming velocity, reduced mass, and stiffness.

6 Minimal GLSL contact state

In Python it is useful to store extra diagnostic values. In GLSL the goal is to store only the persistent values that cannot be safely reconstructed from the current frame.

The first implementation assumes:

- natural contacts begin outside collision,
- contacts that exist at frame zero are always rebound,
- A_0 is computed once per contact from incoming velocity and the force law,
- if a starting overlap has $A_P > A_0$, then A_0 is promoted to the current penetration area,
- one shader invocation owns one source particle and writes only that particle's state.

Under those assumptions, each source particle needs one contact-state slot for each particle contact slot in `ccs`. The state is source-owned, so a physical pair can have two mirrored records:

$$P_i.\text{gcs} \rightarrow j \quad \text{and} \quad P_j.\text{gcs} \rightarrow i.$$

The minimum persistent data is:

- whether the slot is active,
- target particle index,
- phase: compression or rebound,
- source velocity at first contact,
- target velocity at first contact,
- contact normal from the source particle to the target particle,
- zero-velocity penetration area A_0 ,
- zero-velocity center distance.

Everything else can be recomputed from current particle state:

- current penetration area A_P ,
- current center distance,
- compression fraction A_P/A_0 ,
- velocity prediction,
- current relative velocity,
- reduced mass,
- whether the contact has exited.

A clear aligned GLSL structure is:

```
struct gcontact {
    uvec4 ids;           // x=active, y=target, z=phase, w=unused
    vec4 startVel;      // xy=source first-contact velocity,
                        // zw=target first-contact velocity
    vec4 geom;          // x=normal_x, y=normal_y,
                        // z=A_zero, w=center_distance_zero
};
```

The particle structure then stores:

```
gcontact gcs[12];
```

The twelve slots match the existing twelve `ccs` particle-contact slots. Wall state can be added later with four additional slots if wall dynamics are moved into the same source-owned particle state.

The memory cost is:

$$16 \text{ bytes} + 16 \text{ bytes} + 16 \text{ bytes} = 48 \text{ bytes per contact.}$$

For twelve contacts:

$$12 \times 48 = 576 \text{ bytes per particle.}$$

This memory buys persistent first-contact velocity, phase, and A_0 . Those are the values needed to prevent a stateless GLSL model from locking at the zero-velocity turnaround point.

7 Assumptions

1. The collision model must remain an overlap-geometry model. Collision state is determined from particle and wall overlap geometry, not from an unrelated force proxy.
2. Capture report files, named `CapXXXXXX.rpt`, are written to `C:\_DJ\gPCDDData\examples`.
3. The simulation must remain parallel friendly. A source particle may read target-particle information, but it should write only to source-owned storage.
4. Particle velocities in `cfg` files represent the incoming velocity before turnaround. They define the particle's initial momentum for the collision.
5. If particles start at $\mathbf{t} = 0$ already overlapping, the starting collision is treated as post-turnaround rebound. The particles are assumed to have entered contact in the past, penetrated to the zero-velocity point, and then rebounded to the current overlap state.
6. Starting wall collisions follow the same rule as starting particle-particle collisions: a wall overlap at $\mathbf{t} = 0$ is treated as rebound, with the incoming first-contact velocity reconstructed from the current wall normal velocity when needed.
7. The zero-velocity point is the point of maximum penetration where the normal component of relative velocity changes from inbound/compression to outbound/rebound.

8. The current Neo model computes the zero-velocity penetration from the force law

$$F(\alpha) = Q\alpha^2$$

and the turnaround depth

$$\alpha_0 = \left(\frac{3\mu v_{in}^2}{2Q} \right)^{1/3}$$

for particle-particle contacts. For wall contacts, the particle mass is used in place of reduced mass.

9. Collision stiffness, Q , is global for now and is configured by `collision_stiffness_q`. Per-particle stiffness is a possible future extension, but it is not part of the current model.
10. The old parameters `momentum_per_area`, `inverse_square_softening`, and `zero_velocity_overlap_fraction` are obsolete for Neo dynamics.
11. Contact state is source-owned. For GPU parity with Python, each source particle needs contact-state memory that stores target or wall id, contact type, phase, first-contact velocities, first-contact normal, A_0 , and zero-center distance.
12. Particle-particle and wall contacts can share one contact-state structure. The contact type distinguishes particle contacts from wall contacts.
13. The old `ccs` and `bcs` contact lists are obsolete once unified Neo contact state is used.
14. The current GLSL replacement should be named `NeoDynamics.glsl` and should fully replace `GeoDynamics.glsl`. The new GLSL must use the Neo force-law model, not the old configured-overlap-fraction model.
15. The binary particle data structure should remove obsolete collision fields. If stiffness remains global, particle binary records do not need per-particle stiffness fields.
16. `Base.py` owns particle storage, contact detection, contact-state memory, reporting, and application of velocities returned by the dynamics model.
17. `NeoDynamics.py` owns collision calculations: zero-velocity overlap, phase-dependent velocity prediction, rebound clamping, and final rebound velocity calculations.
18. `SimCalc.py` owns simulation-specific lifecycle processes such as reservoir release, outlet handling, and periodic boundary wrapping. These are not raw collision dynamics.